

Дисциплина «Мобильная разработка на платформе .NET MAUI»

ЛЕКЦИЯ 1

Введение в мобильную разработку и .NET MAUI

Цель лекции

Сформировать у студентов целостное представление о мобильной разработке как отдельной области программной инженерии и определить место платформы .NET MAUI в современной экосистеме .NET.

План лекции

1. Что такое мобильное приложение
2. Мобильные платформы
3. Кроссплатформенная разработка
4. Архитектура .NET MAUI

1. Что такое мобильное приложение

Мобильное приложение — это программное обеспечение, спроектированное и оптимизированное для работы на смартфонах, планшетах и других портативных устройствах с сенсорным экраном. В отличие от настольного приложения, мобильное приложение изначально создаётся с учётом жёстких ограничений устройства: небольшого экрана, автономного питания, нестабильной сети и постоянного контроля со стороны операционной системы. Понимание этих особенностей — обязательное условие для проектирования качественного мобильного продукта, поэтому прежде чем переходить к инструментам разработки, необходимо разобрать саму природу мобильной платформы.

1.1. Особенности смартфонов и планшетов

Смартфоны и планшеты принципиально отличаются от персональных компьютеров по форм-фактору и назначению. Это личное, всегда находящееся при пользователе устройство, а не совместно используемая рабочая станция. Ключевые особенности:

- небольшой экран с высокой плотностью пикселей и возможностью поворота (портретная/альбомная ориентация);
- огромное разнообразие аппаратных конфигураций (диагонали экрана, чипсеты, объём памяти) — так называемая фрагментация устройств;
- широкий набор встроенных датчиков: GPS/ГЛОНАСС, акселерометр, гироскоп, датчик освещённости, приближения, камера, микрофон, сканер отпечатка пальца, модуль распознавания лица;
- несколько модулей связи одновременно: Wi-Fi, сотовая сеть, Bluetooth, NFC;
- устройство персонализировано под одного пользователя и почти всегда находится при нём, что открывает сценарии, недоступные настольным приложениям (геолокация, push-уведомления, биометрия).

1.2. Сенсорный интерфейс

Основным способом взаимодействия с мобильным приложением является сенсорный экран, а не мышь и клавиатура. Это принципиально меняет подход к проектированию интерфейса:

- базовые жесты — касание (tap), двойное касание, долгое нажатие (long press), проведение (swipe), сведение/разведение пальцев (pinch-to-zoom), перетаскивание (drag);
- отсутствует точный указатель, как у мыши, — палец существенно менее точен, поэтому элементы управления должны иметь достаточный размер области нажатия (обычно не менее 44×44 условных пикселей);
- экранная клавиатура занимает значительную часть экрана при вводе текста и должна учитываться при проектировании раскладки;
- жесты платформозависимы: например, жест «назад» на Android реализован иначе, чем свайп-навигация на iOS;
- необходимо предусматривать поддержку специальных возможностей (screen reader, голосовое управление, увеличенный текст) для пользователей с ограничениями.

1.3. Ограниченные ресурсы

Несмотря на стремительный рост производительности современных мобильных устройств, их аппаратные ресурсы остаются существенно ограниченнее, чем у настольных компьютеров и серверов. Разработчик обязан учитывать:

- ограниченный объём оперативной памяти — операционная система может принудительно выгружать неактивные приложения при её нехватке;
- ограниченную вычислительную мощность процессора и необходимость экономить её ради отзывчивости интерфейса;
- ограниченный и часто небыстрый объём постоянной памяти (хранилища);
- тепловые ограничения — длительная нагрузка на процессор вызывает троттлинг (снижение частоты) для предотвращения перегрева.

Следствие для разработки — экономное использование памяти, отложенная (ленивая) загрузка данных и изображений, кэширование и аккуратная работа с длительными вычислениями, которые следует выполнять асинхронно и в фоновых потоках.

1.4. Особенности сети

Сетевое подключение мобильного устройства принципиально отличается от подключения настольного компьютера: оно нестабильно, часто переключается между сотовой сетью (3G/4G/5G) и Wi-Fi, может кратковременно пропадать (лифт, метро, роуминг) и, как правило, ограничено по объёму трафика или его стоимости. Из этого вытекают требования к архитектуре мобильного приложения:

- приложение должно корректно работать (или хотя бы не аварийно завершаться) при отсутствии сети — предусматривать offline-режим и локальное кэширование данных;
- необходимы механизмы синхронизации данных при восстановлении соединения и политики повторных попыток запроса (retry policy);
- сетевые запросы следует делать компактными — использовать сжатие, страничную загрузку (pagination), избегать избыточных данных, чтобы экономить трафик и заряд батареи;
- задержки (latency) в мобильных сетях обычно выше, чем в локальной сети, что нужно учитывать при проектировании отзывчивого интерфейса.

1.5. Энергопотребление

Автономность работы от батареи — один из важнейших ограничивающих факторов мобильной разработки, которого попросту не существует для десктопных приложений, постоянно подключённых к сети питания. Заряд батареи расходуют экран, модули связи (особенно сотовая сеть и GPS), процессор и фоновые задачи. Поэтому операционные системы Android и iOS жёстко регламентируют работу приложений в фоне и агрессивно завершают процессы, которые расходуют ресурсы впустую.

Разработчику мобильного приложения необходимо: минимизировать использование геолокации и датчиков, когда в них нет прямой необходимости; группировать сетевые запросы, а не выполнять их поодиночке; использовать штатные платформенные механизмы фоновой работы (например, WorkManager на Android или Background Tasks на iOS) вместо постоянно работающих в фоне служб; избегать удержания «wake lock», не позволяющего устройству переходить в спящий режим.

1.6. Жизненный цикл мобильного приложения

В отличие от настольного приложения, которое работает до тех пор, пока пользователь сам его не закроет, жизненным циклом мобильного приложения полностью управляет операционная система. Она может в любой момент

перевести приложение в фон, приостановить или полностью завершить его процесс, чтобы освободить память для активного приложения на переднем плане. Типичные состояния мобильного приложения:

- Not Running — приложение не запущено и не находится в памяти;
- Foreground / Running — приложение активно, находится на экране и взаимодействует с пользователем;
- Background — приложение не видно на экране, но процесс ещё существует и может выполнять ограниченную фоновую работу;
- Suspended — процесс приложения находится в памяти, но не выполняет код, ожидая возможного возврата в фон/foreground;
- Terminated — процесс приложения полностью завершён операционной системой либо самим приложением.

Ключевое следствие для разработчика — приложение обязано корректно сохранять и восстанавливать своё состояние при переходах между этими стадиями, поскольку пользователь ожидает, что после возврата к приложению он окажется ровно в той точке, где его оставил. В .NET MAUI для реакции на подобные переходы предусмотрены соответствующие события жизненного цикла (например, OnStart, OnSleep, OnResume в классе App, а также события конкретного окна) — подробно они будут рассмотрены в следующих лекциях.

1.7. Различия между мобильным и настольным приложением

Обобщим рассмотренные особенности в виде сравнительной таблицы, которая наглядно показывает, почему мобильная разработка не может рассматриваться как простое «уменьшение» десктопной.

Критерий	Мобильное приложение	Настольное приложение
Способ ввода	Сенсорный экран, жесты, голос	Клавиатура, мышь
Размер экрана	Малый, переменный, есть повороты	Большой, чаще фиксированная ориентация
Управление жизненным циклом	Полностью контролируется ОС	Контролируется пользователем
Ресурсы	Строго ограничены	Существенно больше и предсказуемое
Сеть	Нестабильная, лимитированная, разные каналы	Как правило, стабильная (Ethernet/Wi-Fi)
Питание	Автономное, батарея — критичный ресурс	Чаще от сети, ограничение менее критично

Распространение	Магазин приложений, модерация, песочница	Установочные файлы, более широкие права
Модель разрешений	Гранулярные разрешения в реальном времени	Как правило, разрешения на уровне ОС/учётной записи

Таблица 1. Сравнение мобильного и настольного приложений

2. Мобильные платформы

Рынок мобильных устройств сегодня фактически поделён между двумя доминирующими операционными системами — Android и iOS, при этом экосистема .NET MAUI также позволяет создавать приложения для Windows и macOS (через технологию Mac Catalyst), что делает платформу по-настоящему кроссплатформенной не только для мобильных, но и для настольных сценариев.

2.1. Обзор платформ

- Android — операционная система компании Google на базе ядра Linux, открытая и максимально фрагментированная экосистема: множество производителей устройств, версий ОС и надстроек над интерфейсом. Приложения нативно разрабатываются на Kotlin или Java, распространяются преимущественно через Google Play.
- iOS — операционная система компании Apple, работающая исключительно на собственных устройствах компании (iPhone, iPad). Экосистема закрытая и строго контролируемая: единый производитель устройств, жёсткая модерация публикации приложений в App Store. Нативная разработка ведётся на языке Swift (ранее — Objective-C).
- Windows — настольная операционная система Microsoft; в контексте .NET MAUI выступает как одна из целевых платформ для десктопных приложений (через WinUI 3), что позволяет из единой кодовой базы получить полноценное приложение для Windows наравне с мобильными платформами.
- macOS / Mac Catalyst — операционная система Apple для настольных компьютеров Mac. Mac Catalyst — технология, позволяющая запускать приложения, изначально написанные под iPad, на macOS практически без изменений в коде, адаптируя интерфейс под работу с мышью и клавиатурой.

2.2. Типы приложений

При выборе технологии разработки принято различать четыре основных подхода к созданию приложения для мобильных устройств. Каждый из них по-разному распределяет соотношение «производительность/доступ к платформе» и «скорость разработки/стоимость поддержки».

- **Нативное приложение (Native)** — разрабатывается на официальном языке и SDK конкретной платформы (Kotlin/Java для Android, Swift для iOS). Обеспечивает максимальную производительность и полный доступ ко всем возможностям устройства, но требует отдельной реализации и отдельной команды под каждую платформу.
- **Кроссплатформенное приложение (Cross-platform)** — разрабатывается на едином языке и из единой кодовой базы (например, C# в .NET MAUI, Dart во Flutter), при этом компилируется в нативный код или использует нативные элементы интерфейса каждой платформы. Позволяет существенно сократить дублирование кода бизнес-логики при сохранении близкого к нативному быстродействия и внешнего вида.
- **Веб-приложение (Web / PWA)** — работает внутри браузера и не требует установки из магазина приложений; одна и та же реализация работает на любой платформе с браузером. Ограничено в доступе к аппаратным возможностям устройства и, как правило, требовательно к наличию подключения к интернету.
- **Гибридное приложение (Hybrid)** — веб-технологии (HTML/CSS/JS), упакованные в нативный контейнер и отображаемые через встроенный веб-компонент (WebView). Позволяет быстро адаптировать существующий веб-код под мобильные магазины приложений, однако производительность и характер взаимодействия сильно зависят от используемого веб-движка.

3. Кроссплатформенная разработка

Каждый из рассмотренных подходов имеет свою нишу применения. Ниже приведено сопоставление наиболее распространённых вариантов создания мобильного приложения с точки зрения используемого языка программирования, а также ключевых преимуществ и ограничений.

Подход	Язык	Плюсы	Минусы
Android Native	Kotlin / Java	Полный доступ к возможностям Android	Отдельная реализация под платформу
iOS Native	Swift	Полный доступ к возможностям iOS	Отдельная реализация под платформу

.NET MAUI	C#	Общий код для нескольких платформ	Ограничения уровня абстракции
Web / Hybrid	JS / TS	Единый web-стек разработки	Зависимость от web-исполняющей среды

Таблица 2. Сравнение подходов к кроссплатформенной разработке

Как видно из таблицы, нативная разработка даёт максимум возможностей ценой удвоения (а для нескольких платформ — умножения) усилий команды. Веб- и гибридные решения дают максимальную переносимость кода, но платят за это ограниченным доступом к платформе и зависимостью от производительности исполняющей веб-среды. .NET MAUI занимает промежуточную и во многих отношениях наиболее сбалансированную позицию: единая кодовая база на C#, общая бизнес-логика и значительная часть пользовательского интерфейса, при этом критичные по производительности и внешнему виду элементы управления отображаются нативными компонентами конкретной платформы, а не эмулируются внутри веб-контейнера.

4. Архитектура .NET MAUI

.NET MAUI (Multi-platform App UI) — это кроссплатформенный фреймворк для создания нативных мобильных и настольных приложений на языке C# и XAML, являющийся идейным и технологическим развитием более раннего фреймворка Xamarin.Forms и входящий в единую платформу .NET начиная с .NET 6. Принципиальная идея .NET MAUI — «один проект, много платформ»: разработчик ведёт единую кодовую базу и единый проект (Single Project), а специфичный для платформы код и ресурсы подключаются условно, через конфигурацию целевых сред (target frameworks) в рамках этого же проекта.

Архитектуру .NET MAUI удобно представить в виде последовательности слоёв, каждый из которых опирается на предыдущий и в конечном счёте транслируется в вызовы нативного API конкретной операционной системы:

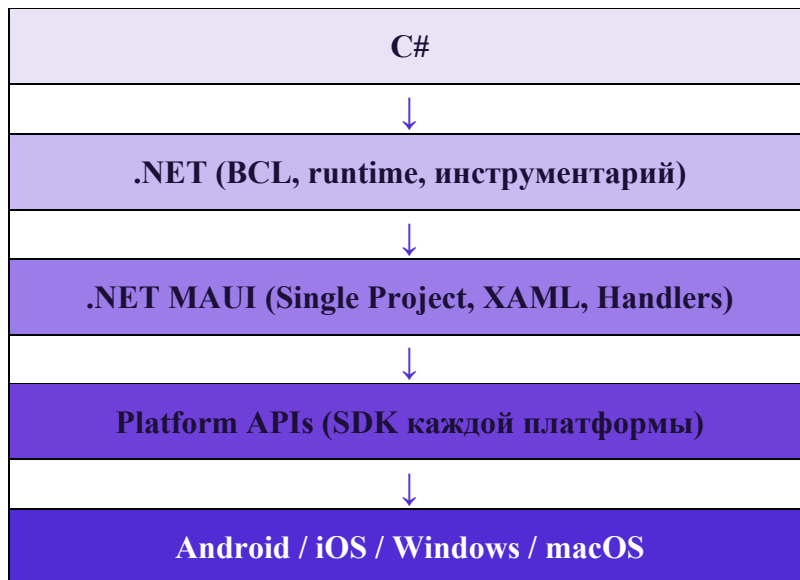


Схема 1. Слои архитектуры .NET MAUI

Рассмотрим каждый слой подробнее.

- C# — язык программирования, на котором пишется бизнес-логика приложения, модели представления (ViewModel) и, при использовании XAML, обработчики событий и code-behind.
- .NET — единая платформа и базовая библиотека классов (Base Class Library), общая для всех типов приложений на .NET (веб, десктоп, облако, мобильные устройства), а также инструменты сборки, единый SDK и менеджер пакетов NuGet.
- .NET MAUI — кроссплатформенный UI-фреймворк, который поверх .NET предоставляет: единую модель проекта (Single Project), общий набор элементов управления и разметку на XAML, а также архитектуру Handlers — набор классов-обработчиков, которые связывают кроссплатформенный элемент управления с его нативной реализацией на конкретной платформе (более лёгкий и производительный подход по сравнению с архитектурой Renderer'ов, использовавшейся в Xamarin.Forms).
- Platform APIs — нативные SDK конкретной платформы (Android SDK, iOS SDK, WinUI 3, Mac Catalyst API), которые фреймворк .NET MAUI использует «под капотом» для отображения реальных, полностью нативных элементов управления и обращения к аппаратным и системным возможностям устройства.
- Android / iOS / Windows / macOS — конечные целевые операционные системы, на которых в итоге выполняется и с которыми взаимодействует собранное приложение.

Такая слоистая архитектура позволяет достичь главной цели кроссплатформенной разработки на .NET MAUI: разработчик пишет код один раз на C#, а на выходе получает по-настоящему нативные приложения для Android, iOS, Windows и macOS, использующие подлинные элементы управления и API каждой платформы, а не их эмуляцию.

Заключение

В рамках данной лекции были рассмотрены фундаментальные отличия мобильной разработки от разработки настольных приложений: специфика мобильных устройств и сенсорного взаимодействия, жёсткие ограничения по ресурсам, сети и энергопотреблению, а также управляемый операционной системой жизненный цикл приложения. Были разобраны основные мобильные и настольные платформы, поддерживаемые современными кроссплатформенными технологиями, и проведено сопоставление нативного, кроссплатформенного, веб- и гибридного подходов к разработке. Наконец, рассмотрена архитектура .NET MAUI как современного решения от Microsoft, позволяющего создавать нативные приложения для Android, iOS, Windows и macOS из единой кодовой базы на языке C#. Полученные представления образуют фундамент для дальнейшего изучения курса, в котором подробно будут рассмотрены установка и настройка окружения разработки, структура проекта .NET MAUI, разметка интерфейса на XAML и архитектурные шаблоны построения мобильных приложений.

Контрольные вопросы

1. Какие особенности смартфонов и планшетов необходимо учитывать при проектировании мобильного приложения?
2. Почему сенсорный интерфейс требует иного подхода к проектированию элементов управления по сравнению с интерфейсом «мышь + клавиатура»?
3. Какие ограничения ресурсов устройства влияют на архитектуру мобильного приложения?
4. Какие приёмы позволяют мобильному приложению корректно работать при нестабильном сетевом соединении?
5. Почему энергопотребление считается критическим фактором именно для мобильной, а не для настольной разработки?
6. Перечислите основные состояния жизненного цикла мобильного приложения и объясните переходы между ними.
7. В чём принципиальные различия между нативным, кроссплатформенным, веб- и гибридным подходами к разработке приложений?
8. Опишите архитектуру .NET MAUI по слоям — от языка C# до конкретной платформы.
9. Чем архитектура Handlers в .NET MAUI отличается от архитектуры Renderer'ов в Xamarin.Forms?

Рекомендуемая литература

- Официальная документация Microsoft по .NET MAUI (learn.microsoft.com).
- Официальные руководства по разработке для Android (developer.android.com) и iOS (developer.apple.com).
- Материалы по принципам проектирования мобильных пользовательских интерфейсов (Material Design, Human Interface Guidelines).
- Учебные материалы по архитектурным шаблонам MVVM для .NET-приложений.